Manual de usuario — UruTurn!

1. ¿Para qué sirve el sistema?

UruTurn! permite consultar establecimientos y profesionales, solicitar turnos, seguir el estado de una solicitud y consultar las facturas generadas.

Importante: esta versión todavía no tiene una pantalla propia para clientes. Las operaciones se hacen desde Postman, también es posible acceder a los servicios a través del navegador en http://localhost:8000/docs. Este manual se centra en explicar cómo usar la interfaz de Postman.

2. Abrir el sistema

Una persona encargada debe iniciar el sistema con Docker Compose:

```
docker compose up -d --build
```

Cuando los servicios estén iniciados:

- **API y operaciones:** Postman
- **Administración técnica de la base de datos:** http://localhost:8080/

Para el uso habitual, utilizar Postman. Para revisar directamente la base de datos, ingresar al puerto 8080 desde el navegador. phpMyAdmin es una herramienta administrativa; no se necesita para solicitar ni consultar turnos.

Si la página no abre, pedir a la persona encargada que verifique que Docker esté funcionando y ejecute:

```
docker compose ps
```

3. Conceptos del sistema

Estas son las entidades que modela el sistema y algunas observaciones a tener en cuenta:

- **Cliente** (email, nombre, teléfono): se identifica por su email. No se provee una API para la gestión de clientes, su creación y modificación se desprende de las reservas de turnos que se realicen bajo una misma dirección de correo. Un cliente se relaciona con las reservas y facturas que estén registradas con su mail.
- **Establecimiento** (id_establecimiento, nombre_comercial, direccion, telefono, correo_electronico, horario_apertura, horario_cierre): se identifica por su ID. A un establecimiento pueden pertenecer múltiples profesionales (personal) y recibe múltiples reservas.
- **Personal** (id_personal, id_establecimiento, nombre, apellido, email, telefono, cargo, especialidad, costo_consulta, duracion_atencion, activo): se identifica por su ID. Un profesional pertenece a un único establecimiento, indicado por su id_establecimiento, y atiende múltiples reservas. El atributo “activo” es binario, un valor 0 implica que el profesional ya no se encuentra en actividad, se recomienda no

borrar profesionales, basta con establecer su valor activo a 0 para mantener un registro y evitar conflictos con la base de datos.

- **Reserva** (id_reserva, fecha_reservado, email_solicitante, telefono_solicitante, id_personal, id_establecimiento, fecha_turno, hora_turno, estado_reserva, id_personal_ocupado): se identifica por su ID, corresponde a un único cliente indicado por su email_solicitante, y a un profesional (id_profesional). El sistema maneja internamente el estado de las solicitudes, los estados más relevantes son:

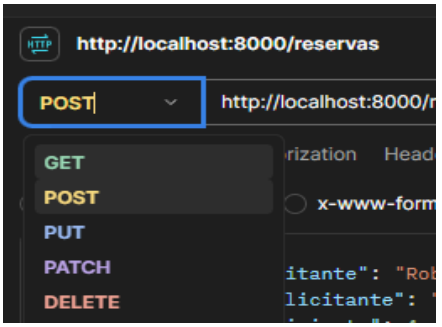
Estado	Significado
`PENDIENTE`	La solicitud se recibió y espera validación.
`AGENDADO`	El turno fue aceptado y quedó agendado.
`RECHAZADO_SOLICITUD_NO_VALIDA`	La fecha, el horario o los datos del profesional no cumplen las condiciones.
`RECHAZADO_TURNO_OCUPADO`	El profesional ya tiene un turno en ese horario.
`CANCELADO`	El turno fue cancelado.
`ATENDIDO`	La fecha y hora del turno pasaron y quedó registrado como atendido.
`FACTURADO`	El turno atendido ya fue incluido en una factura.

- **Factura** (id_factura, email_cliente, periodo, fecha_emision, monto_total): se identifica por su ID. Corresponde a un cliente (email_cliente), tiene múltiples items, y su periodo refiere al mes y año. El monto total se calcula sumando el precio de todos sus respectivos items.
- **Factura_item** (id_item, id_factura, id_reserva, id_personal, nombre_profesional, descripcion, precio_unitario): se identifica por su ID. Corresponde a una sola factura (factura_id) y una sola reserva (reserva_id). Se relaciona con el profesional correspondiente a la reserva, el precio unitario equivaldrá al costo de consulta del profesional.

4. Uso general de una API

Se utiliza Postman para enviar los mensajes HTTP a las distintas APIs, las APIs en cuestión permiten la alta, baja, modificación y consulta de: establecimientos, personal, reservas, facturas.
El primer paso es seleccionar el verbo HTTP según la acción que se quiera realizar:

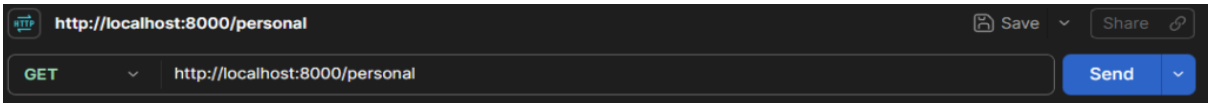
- GET para consultar
- POST para crear
- PUT para modificar
- DELETE para eliminar



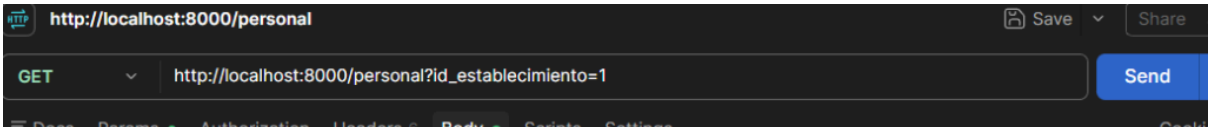
Se parte de la ruta “http://localhost:8000/”.
Para especificar a cuál de las APIs enviar el mensaje, se agrega al final de la ruta el nombre de la API deseada (establecimientos/personal/reservas/facturas).
Para enviar los mensajes, presionar el botón “Send”.

Cómo usar GET

Seleccionar el verbo GET, ingresar la API deseada en la ruta, si se quiere, y enviar. Es posible filtrar por ID agregando /(id) al final de la ruta (reemplazar “(id)” por el número de ID deseado). La API de personal ofrece adicionalmente filtros por establecimiento y por cargo, para utilizarlos, agregar “?establecimiento=(id_del_establecimiento)” o “?cargo=(nombre_del_cargo)”, nótese que el filtro por cargo NO es sensible a mayúsculas ni tildes.

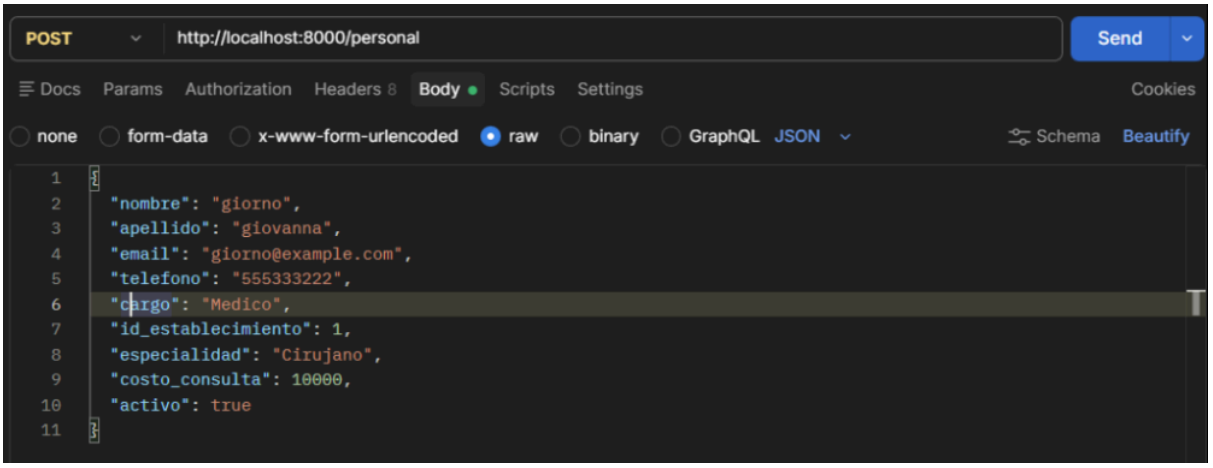






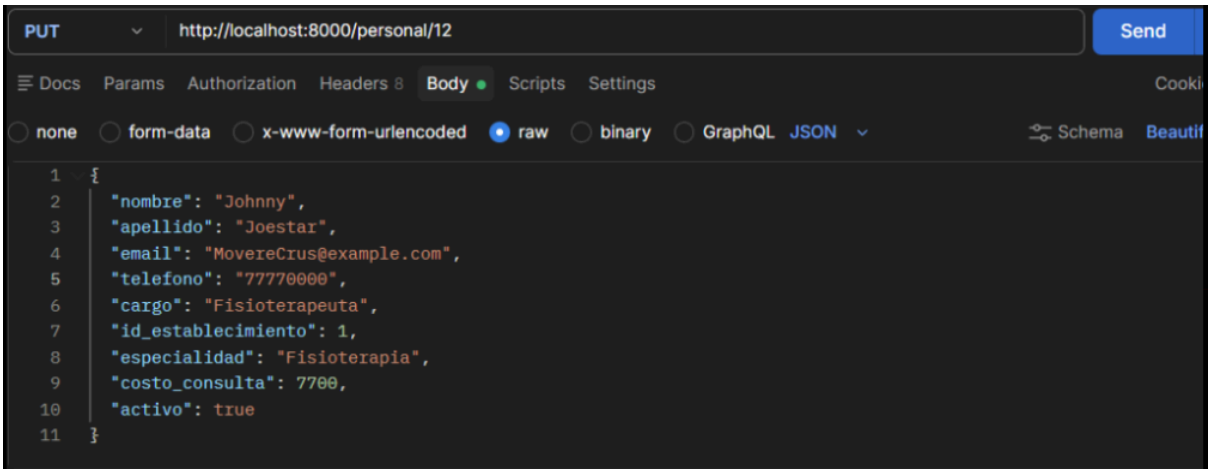
Cómo usar POST

Seleccionar el verbo POST, ingresar la API en la ruta, seleccionar “Body” y marcar la opción “raw”, ingresar el cuerpo del archivo JSON (body) con los datos de la nueva entidad y enviar. Se puede consultar el cuerpo de los payload JSON para cada API desde el navegador ingresando a http://localhost:8000/docs.



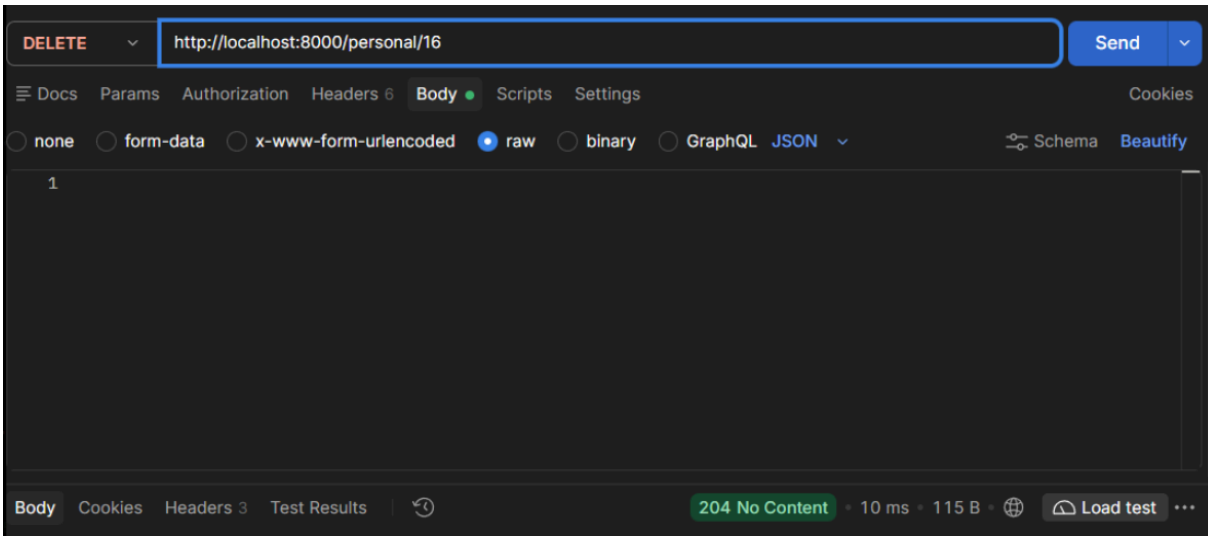
Cómo usar PUT

Seleccionar el verbo PUT, ingresar en la ruta la API y el número de la ID que corresponda a la entidad que se desea actualizar, seleccionar “Body” y marcar “raw”, ingresar el body correspondiente al PUT según la API (consultar en http://localhost:8000/docs) y enviar.



Cómo usar DELETE

Seleccionar el verbo DELETE, ingresar en la ruta la API y el número de ID de la entidad a ser eliminada y enviar.



Scripts de ayuda

El proyecto incluye scripts en Bash (archivos `.sh`) para automatizar la ejecución de solicitudes HTTP mediante `curl`, probar la API y facilitar tareas comunes sin necesidad de herramientas gráficas.

A continuación se detallan las rutas, comandos y scripts disponibles:

Script / Archivo .sh	Comando / Ruta	Descripción
`scripts/probador.sh`	`bash scripts/probador.sh`	Ejecuta peticiones curl automatizadas para verificar los endpoints principales de la API.
`scripts/cargar_datos.sh`	`bash scripts/cargar_datos.sh`	Carga datos iniciales de prueba (establecimientos, personal y turnos) vía la API.

5. Devoluciones

Al enviar un mensaje HTTP a una API, esta responderá con un código HTTP. Aquí las devoluciones comunes del sistema y su significado.

- 200: indica que la operación ha sido exitosa.
- 201: creado, indica que la entidad se dio de alta correctamente.
- 202: aceptado, devuelve al dar de alta una reserva con datos válidos.
- 204: sin contenido, suele indicar que una entidad ha sido eliminada exitosamente.
- 400: indica que el mensaje no es válido, puede significar que se está solicitando una operación conflictiva (por ejemplo, reservar un turno para un profesional inactivo) o que el cuerpo del JSON está incompleto o mal estructurado.
- 404: no encontrado, indica que un dato requerido no existe en la base de datos.
- 409: conflicto, suele indicar que hay un valor de tipo único que se repite al intentar ingresar los datos.
- 500: error interno del servidor.
- 503: servicio no disponible.